

Project Kickstart

AI-driven playbook for project initiation — **10 steps to launch a clean, governed, build-ready code repository**

After Step-00, each step is one Claude command, run in a fresh session **from the kit**. It drafts the step's documents in your **target repo** and opens a PR/MR. The documents become final only when the Product Owner and/or Architect approves and merges it to `main`.

KIT · run commands Control plane Commands, step guides, templates, step registry, role files.	TARGET · registered by Step-00 Your project clone Every git operation and every produced document lands here.	GIT HOST · GitHub or others Review and merge One branch and one PR/MR per step. Merge to <code>main</code> = final.
---	---	---

What each step produces and waits for

Every step's document moves through the same four states. On a branch it's a draft; on `main` it's final.

HUMAN · starts Initiate Run from the kit; nothing in the target yet <code>/proj-init-bootstrap</code>	CLAUDE · drafts Draft On <code>init/<step></code> in the target <code>/proj-init-<step></code>	HUMAN · approves Review Pushed after your yes, PR/MR open <code>gh pr create</code>	HUMAN · merges Final Merged after the self-review checklist <code>merge to main</code>
--	---	--	---

A step won't start until its inputs below are on `main`; steps 02–09 also need `CONTRIBUTING.md`.

Step	Document	Purpose	Needs on main
01	<code>CONTRIBUTING.md</code>	Governance and tooling rules	Step-00 registration
02	<code>PRODUCT.md</code>	Product concept	<code>CONTRIBUTING.md</code>
03	<code>PRD.md</code>	Product requirements	<code>PRODUCT.md</code>
04	<code>ARCHITECTURE.md</code>	System design	<code>PRD.md</code>
05	<code>TECH-STACK.md</code>	Approved technologies	<code>PRD.md</code> , <code>ARCHITECTURE.md</code>
06	<code>ENGINEERING-RULES.md</code> + <code>CLAUDE.md</code>	Coding rules; AI assistant instructions	<code>ARCHITECTURE.md</code> , <code>TECH-STACK.md</code>
07	<code>README.md</code>	Project entry point	All documents from 02–06
08	<code>BACKLOG.md</code>	Initial backlog	02–06 + <code>README.md</code>
09	<code>CONTRIBUTING.md</code>	Permanent contribution core	<code>BACKLOG.md</code>

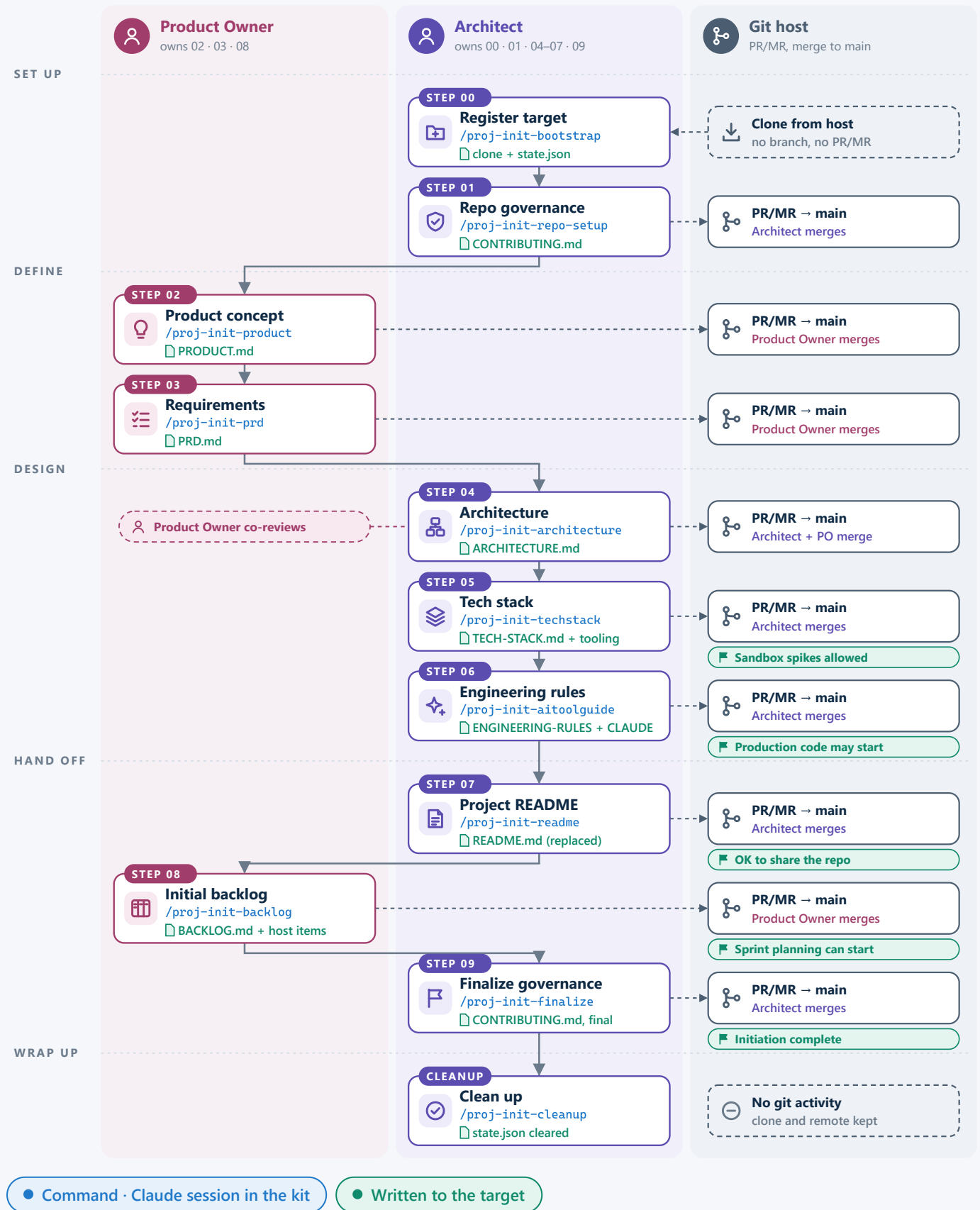
Start every step the same way

TERMINAL	<code>cd <kit-folder></code> <code>claude</code>	# from the kit folder – never from the target clone
CLAUDE	<code>/proj-init-<step></code>	# e.g. <code>/proj-init-product</code>

One step, one session. For the next step, run `/clear` or start a new `claude` session. Each step syncs `main` and checks status itself; run `/proj-init-doc-status` any time to see what's next.

The whole flow, by who does what

Read top to bottom. Each step sits in the lane of the role that owns it; an elbow in the arrow is a handoff. The right column shows each PR/MR, who merges it, and what that merge unlocks.



Docs land under `docs/` in the target; `README.md` and `CONTRIBUTING.md` stay at the root. Solo team? Wear the hat of the lane the step sits in.

Step by step

Run every command from the kit folder, one per fresh session. Steps 01–09 share one runner: resolve the target, fast-forward `main`, check status and preconditions, wait for your **yes**, branch, interview you one question at a time, draft, commit, then push and open the PR/MR only after a second **yes**.

Each step card is tagged with where it runs and who acts:

- Kit** Runs in the kit repo. You start each Claude command here, and the guides and templates load from here.
- Target** Lands in your project clone: the step's branch, commits, and documents.
- Human** Needs a person to act: confirm, review, approve, or merge. The step's owner (Product Owner and/or Architect) does this.

00

Register the target repo

Kit

Human

Clones your project repo and records where it lives, so every later step operates on that clone. Nothing is committed or pushed.

CLAUDE	/proj-init-bootstrap
ASKS	The target's git URL, and an empty or non-existent local folder to clone into.
RUNS	<code>node scripts/bootstrap-target-repo.mjs --target <folder> --url <git-url></code> as a dry run, then again with <code>--apply</code>
WRITES	The clone at <code><folder></code> , plus <code>.proj-init/state.json</code> in the kit (gitignored, local to this machine)
GATE	■ You review the dry run before <code>--apply</code> . Only one workspace at a time; replace it only with <code>--force</code> .
GIT	No branch, no commit, no PR/MR.

01

Set up repo governance

Kit

Target

Human

The gate every later step relies on: the branching convention, the commit convention, and the self-review checklist.

CLAUDE	/proj-init-repo-setup
OWNER	Architect · reviewed by Architect
WRITES	<code>CONTRIBUTING.md</code> : permanent rules plus an <code>INITIATION-ONLY</code> fenced block, and branch protection on <code>main</code> if your host and plan allow it
GIT	<code>init/repo-setup</code> · <code>docs: add repo governance baseline (Step-01)</code>
GATE	■ Self-review checklist, then you merge. Every later step stops until <code>CONTRIBUTING.md</code> is on <code>main</code> .

Product concept

What we are building, and why.

CLAUDE	/proj-init-product
--------	--------------------

OWNER	Product Owner · reviewed by Product Owner
READS	CONTRIBUTING.md on main (the Step-01 gate)
WRITES	docs/PRODUCT.md
GIT	init/product · docs: add PRODUCT.md (Step-02)
GATE	■ The Product Owner approves and merges.

03

Product requirements

● Kit

● Target

● Human

Turns the concept into testable requirements.

CLAUDE

/proj-init-prd

OWNER Product Owner · reviewed by Product Owner

READS docs/PRODUCT.md

WRITES docs/PRD.md

GIT init/prd · docs: add PRD.md (Step-03)

GATE ■ The Product Owner approves and merges.

04

Architecture

● Kit

● Target

● Human

System structure and design decisions. The runner loads both the Architect and Product Owner role context.

CLAUDE

/proj-init-architecture

OWNER Architect · reviewed by Architect + Product Owner

READS docs/PRD.md

WRITES docs/ARCHITECTURE.md

GIT init/architecture · docs: add ARCHITECTURE.md (Step-04)

GATE ■ The Architect and Product Owner both approve, then merge.

05

Tech stack

● Kit

● Target

● Human

Locks the approved technologies and adds the build/test/lint commands to CONTRIBUTING.md .

CLAUDE

/proj-init-techstack

OWNER Architect · reviewed by Architect

READS docs/PRD.md , docs/ARCHITECTURE.md

WRITES docs/TECH-STACK.md ; appends the tooling layer to CONTRIBUTING.md and leaves the Step-01 governance layer intact

GIT init/techstack · docs: add TECH-STACK.md and CONTRIBUTING tooling layer (Step-05)

GATE ■ The Architect approves and merges.

AFTER MERGE Sandbox spikes are allowed, on isolated spike branches that never merge to main .

06

Engineering rules

● Kit

● Target

● Human

The coding rules every contributor follows, human or AI. Shared rules live in `ENGINEERING-RULES.md` ; `CLAUDE.md` points to it and adds only what's specific to Claude Code.

CLAUDE

/proj-init-aitoolguide

OWNER

Architect · reviewed by Architect

READS

`docs/ARCHITECTURE.md` , `docs/TECH-STACK.md`

WRITES

`docs/ENGINEERING-RULES.md` and `CLAUDE.md`

GIT

`init/aitoolguide` · docs: add `ENGINEERING-RULES.md` and `CLAUDE.md` (Step-06)

GATE

■ The Architect approves and merges.

AFTER MERGE

Production implementation can start.

07

Project README

● Kit

● Target

● Human

Replaces the target's boilerplate README with a real entry point: setup, environment config, and how to run.

CLAUDE

/proj-init-readme

OWNER

Architect · reviewed by Architect

READS

`PRODUCT.md` , `PRD.md` , `ARCHITECTURE.md` , `TECH-STACK.md` , `ENGINEERING-RULES.md`

WRITES

`README.md` , replacing the existing file with all placeholders removed

GIT

`init/readme` · docs: replace boilerplate README with project README (Step-07)

GATE

■ Every command in the README is tested before commit. The Architect approves and merges.

AFTER MERGE

Required before the repo is shared more widely.

08

Initial backlog

● Kit

● Target

● Host

● Human

Epics and stories derived from the product and requirements docs, seeded into your issue tracker.

CLAUDE

/proj-init-backlog

OWNER

Product Owner · reviewed by Product Owner

READS

The five docs Step-07 reads, plus `README.md`

WRITES

`docs/BACKLOG.md` , plus host work items via `gh` or `az boards` ; with neither CLI available, `PENDING` IDs and manual creation notes

GIT

`init/backlog` · docs: add `BACKLOG.md` – Step-08 initial backlog

GATE

■ You approve the derived backlog before anything is written to the host or to disk. Items are then created one at a time. The Product Owner merges.

AFTER MERGE

Required before sprint planning. Next: Step-09.

09

Finalize governance

● Kit

● Target

● Human

Removes the initiation-only rules from `CONTRIBUTING.md` and hands the team a permanent contribution core.

CLAUDE

/proj-init-finalize

OWNER

Architect · reviewed by Architect

READS

`docs/BACKLOG.md` on `main` (the gate) and `CONTRIBUTING.md` on `main`

WRITES

`CONTRIBUTING.md`: the `INITIATION-ONLY` block is removed (markers included) and a `DEVELOPMENT-PHASE-GOVERNANCE` stub is inserted. Permanent rules and the tooling layer stay verbatim.

GIT

`init/finalize` · `docs: finalize CONTRIBUTING.md for the development phase (Step-09)`

GATE

■ Stops if the `INITIATION-ONLY` markers are missing. Once the Architect merges, initiation is complete.

✓

Clean up the workspace

● Kit

● Human

Unregisters the workspace from the kit. The target clone and its remote are left untouched.

CLAUDE

/proj-init-cleanup

CHECKS

`docs/BACKLOG.md` is on `main` and `INITIATION-ONLY` is gone from `CONTRIBUTING.md`

RUNS

`node scripts/bootstrap-target-repo.mjs --clear`, after your **yes**

WRITES

Deletes `.proj-init/state.json`. Nothing else.

GIT

None.

Utilities, any time

CLAUDE

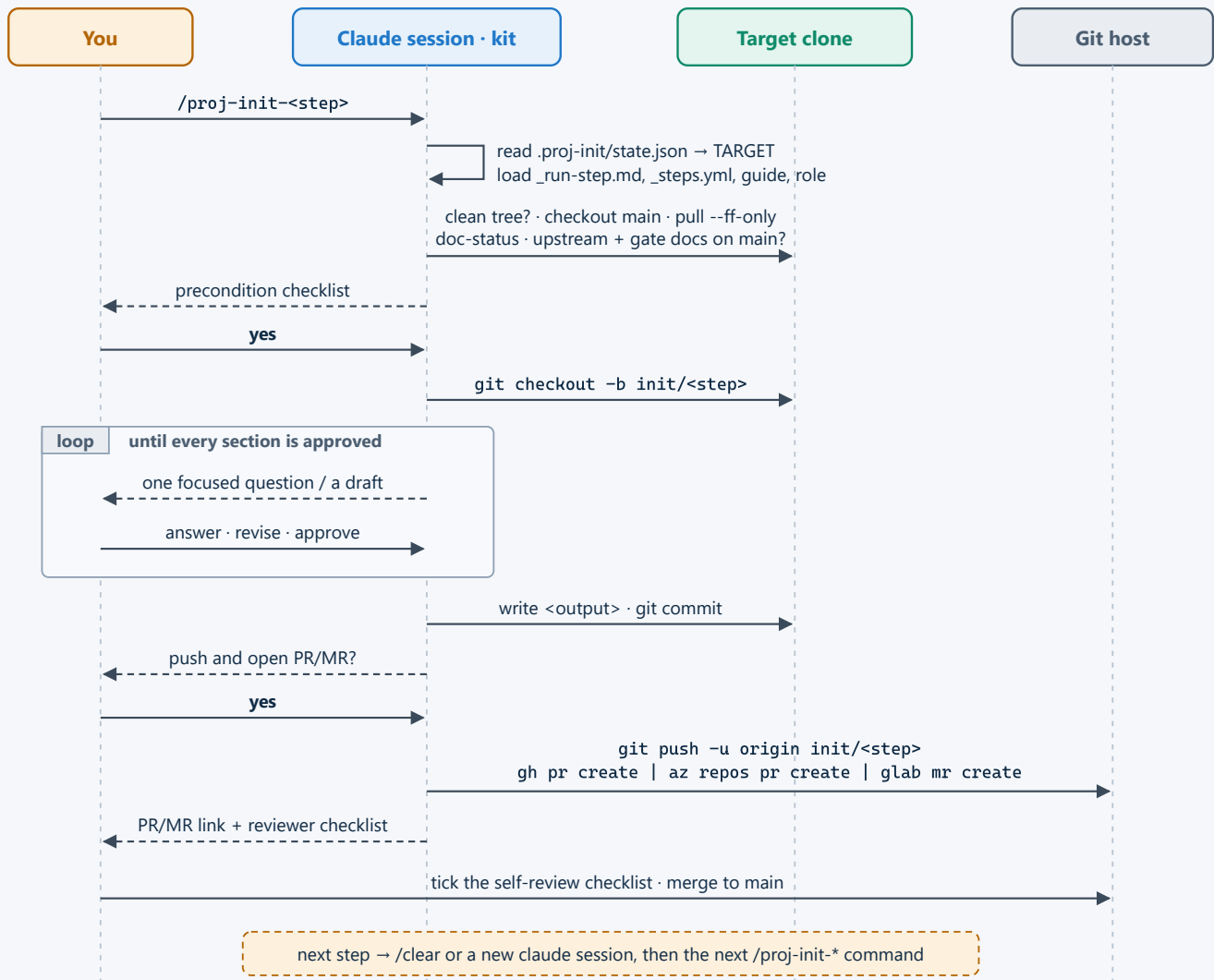
```
/proj-init-doc-status           # read-only: merged, open PRs/MRs, what's next
/proj-init-doc-update <doc>    # e.g. PRD.md → PR/MR + reconciliation checklist
# add --target <folder> to either for a finished or other project
```

Which document to update

If this happens	Run <code>/proj-init-doc-update</code> on
Product scope, goal, or target users change	<code>PRODUCT.md</code>
Requirements or acceptance criteria change	<code>PRD.md</code>
System design or a major architecture decision changes	<code>ARCHITECTURE.md</code>
A library, framework, or tool is added, removed, or version-locked	<code>TECH-STACK.md</code>
Coding conventions, banned patterns, or testing rules change	<code>ENGINEERING-RULES.md</code>
Setup steps, env config, or run commands change	<code>README.md</code>
Epics or stories added, removed, rescoped, or host IDs change	<code>BACKLOG.md</code>

One step up close

What happens between you, the Claude session in the kit, the target clone, and your Git host during any step from 01 to 09.



After the PR/MR opens

From here on it's your job. The command's job ended when the PR/MR opened.

1. **Give it fresh eyes.** Wait at least a few hours, ideally a full day, before reviewing.
2. **Tick the self-review checklist** in the PR/MR description: every required section is covered, no upstream doc changed after you branched, and the change went through a PR/MR, not a direct push.
3. **Merge to `main`.** Merging is what makes it final. Step-04 also needs the Product Owner's sign-off.
4. **Start the next step in a fresh session.** It fast-forwards `main` in the target before it checks that the upstream documents are merged.

Rules that hold at every step

One step, one fresh session. Never run two step commands in the same session. Status is checked first, then only the step you asked for runs.	Run from the kit, write to the target. Commands load guides from the kit and do git work in the registered clone. Kit files are never copied in.
Merge = final. A doc on a branch is a draft; a doc on <code>main</code> is final. There are no draft files and no status flags.	Nothing ships without your yes. Explicit confirmation before branching, before every host write, and before every push.
Never on <code>main</code>. No commits or pushes to <code>main</code> , and no force-pushes to shared branches. Every change goes through a PR/MR.	Upstream first. A step stops if any doc it depends on isn't on <code>main</code> , or if its own output is already final (use <code>/proj-init-doc-update</code> instead).
Same machine, same operator. <code>.proj-init/state.json</code> is local to one machine. To continue elsewhere, re-run Step-00.	Never assume an answer. Claude asks one focused question at a time until no template placeholder is left.

Milestones

AFTER 05 MERGES Spikes Isolated spike branches only, never merged to <code>main</code> .	AFTER 06 MERGES Production code Implementation work can begin.	AFTER 07 MERGES Share the repo The README is the real entry point.	AFTER 08 MERGES Sprint planning Then Step-09 completes initiation.
--	--	--	--

Command cheat sheet

Step	Claude command	Produces in the target	Merged by
00	<code>/proj-init-bootstrap</code>	clone + kit-local <code>.proj-init/state.json</code>	— (you confirm)
01	<code>/proj-init-repo-setup</code>	<code>CONTRIBUTING.md</code> (governance)	Architect
02	<code>/proj-init-product</code>	<code>docs/PRODUCT.md</code>	Product Owner
03	<code>/proj-init-prd</code>	<code>docs/PRD.md</code>	Product Owner
04	<code>/proj-init-architecture</code>	<code>docs/ARCHITECTURE.md</code>	Architect + PO
05	<code>/proj-init-techstack</code>	<code>docs/TECH-STACK.md</code> + tooling layer	Architect
06	<code>/proj-init-aitoolguide</code>	<code>docs/ENGINEERING-RULES.md</code> + <code>CLAUDE.md</code>	Architect
07	<code>/proj-init-readme</code>	<code>README.md</code>	Architect
08	<code>/proj-init-backlog</code>	<code>docs/BACKLOG.md</code> + host work items	Product Owner
09	<code>/proj-init-finalize</code>	<code>CONTRIBUTING.md</code> (init-only removed)	Architect
—	<code>/proj-init-cleanup</code>	nothing; unregisters the workspace	— (you confirm)

Solo / process-enforced: one person may wear both hats; take on the one each step calls for. Source of truth: docs/guides/proj-init/(_steps.yml, _run-step.md, step guides).